

Question - 1

Custom Error

What is the output of this JavaScript code snippet?

```
class CustomError extends Error {
  constructor(message) {
    super(message);
    this.name = "CustomError";
  }
}

function validateData(data) {
  if (!data || typeof data !== "object") {
    throw new CustomError("Invalid data provided");
  }

  if (!data.hasOwnProperty("id") || typeof data.id !== "number") {
    throw new CustomError("Data object is missing an id property");
  }

  return true;
}

const data1 = { id: 42 };
const data2 = "Invalid data";

try {
  validateData(data1);
  console.log("Data1 is valid");
  validateData(data2);
  console.log("Data2 is valid");
} catch (error) {
  if (error instanceof CustomError) {
    console.log(`CustomError: ${error.message}`);
  } else {
    console.log(`Error: ${error.message}`);
  }
}
```

- ☐ Data1 is valid
CustomError: Invalid data provided
- ☐ Data1 is valid
CustomError: Data object is missing an id property
- ☐ Data1 is valid
Error: Invalid data provided
- ☐ No output

Question - 2

SetTimeout

What is the output of this JavaScript code snippet?

```
const numbers = [1, 2, 3, 4, 5];

for (var i = 0; i < numbers.length; i++) {
  setTimeout(() => {
    console.log(`Number: ${numbers[i]}`);
  }, i * 1000);
}
```

- ☐ Number: 1
Number: 2
Number: 3
Number: 4
Number: 5
- ☐ Number: undefined
Number: undefined
Number: undefined
Number: undefined
Number: undefined
- ☐ Number: 5
Number: 5
Number: 5
Number: 5
Number: 5
- ☐ No output will be displayed.

Question - 3

Proxy

```
class Person {
  constructor(name, age) {
    this.name = name;
    this.age = age;
  }

  greet() {
    console.log(`Hello, my name is ${this.name}.`);
  }
}
```

```

const personHandler = {
  get(target, property) {
    if (property === "age") {
      return target[property] + " years old";
    }
    return target[property];
  },
};

const person = new Person("John", 30);
const proxyPerson = new Proxy(person, personHandler);

console.log(proxyPerson.name);
console.log(proxyPerson.age);
proxyPerson.greet();

```

What is the output of this JavaScript code snippet?

- ☐ John
30 years old
Hello, my name is John.
- ☐ John
30
Hello, my name is John.
- ☐ John
30 year old
Hello, my name is John.
- ☐ John
Hello, my name is John
30 years old.

Question - 4

Currying

What is the value of *result* in the following code?

```

function multiply(a) {
  return function(b) {
    return function(c) {
      return a * b * c;
    };
  };
}

const double = multiply(2);
const triple = multiply(3);

const result = triple(double(4)(5))(1);

```

- ☐ Type Error
- ☐ 120
- ☐ 60
- ☐ 240

Question - 5

Equality

What is the output of the following code?

```
const str = "1";
const obj = new String("1");
console.log(obj == "1");
console.log(obj === str);
console.log(obj == str);
```

- ☐ True, False, False
- ☐ True, False, True
- ☐ True, True, True
- ☐ True, True, False

Question - 6

Closures

What is the output of the following code?

```
const series = (...funcs) => (arg) => {
  return funcs.reduce((prev, curr) => curr(prev), arg);
};
const result = series(num => num * 10, num => num + 20, num => num / 2, num => num - 5)(4);
```

- ☐ 20
- ☐ 25
- ☐ 30
- ☐ 0

Question - 7

JavaScript: Objects

Three of the following snippets will create a new JavaScript object with the name "Alex" and age 25. Which one uses incorrect syntax?

☐

```
let person = new Object({name: "Alex", age:25});
```

☐

```
let person = {name: "Alex", age: 25};
```

☐

```
let person = {}  
person.name = "Alex";  
person.age = 25;
```

☐

```
let person;  
person['name'] = "Alex";  
person['age'] = 25;
```

Question - 8

JavaScript: Typecasting Functions I

Match the JavaScript typecasting functions with their respective output.

Function	Output
1. <i>String([])</i>	A. 0
2. <i>String({})</i>	B. -Infinity
3. <i>Number(null)</i>	C. "[object Object]"
4. <i>Number("-Infinity")</i>	D. ""

☐

- 1. A
- 2. B
- 3. D
- 4. C

☐

- 1. D
- 2. C
- 3. A
- 4. B

☐

- 1. A
- 2. D

3. B
4. C

☐ None of the above

Question - 9

Declare variables with const keyword in JavaScript

One way to declare a variable in JavaScript is to use the keyword *const*. Which of the following are true about *const*? Choose one or more.

- ☐ A variable declared with const is always globally scoped.
- ☐ A variable declared with const is always block-scoped.
- ☐ A variable declared with const can be re-declared.
- ☐ A variable declared with const can be assigned to later.

Question - 10

JavaScript: Typecasting Functions II

Match the JavaScript typecasting functions with their respective output.

Function Call	Output
1. <i>Boolean(Infinity)</i>	a. 20
2. <i>Number([20, 30])</i>	b. true
3. <i>Boolean("")</i>	c. NaN
4. <i>Number(["20"])</i>	d. false

- ☐

1-b
2-c
3-d
4-a
- ☐

1-d
2-a
3-b
4-c
- ☐

1-d
2-b
3-a
4-c
- ☐ None of the above

Question - 11

Declare variables with let keyword in JavaScript

One way to declare a variable in JavaScript is to use the keyword *let*. Which of the following are true about *let*? Choose one or more.

- ☐ A variable declared with `let` is always globally scoped.
- ☐ A variable declared with `let` is always block-scoped.
- ☐ A variable declared with `let` can be re-declared.
- ☐ A variable declared with `let` can be assigned to later.

Question - 12

localStorage object

The `localStorage` object is used to store the data locally. Which of the following are true about `localStorage` objects? Select one or more.

- ☐ The data in `localStorage` is stored without an expiration date.
- ☐ `localStorage` allows for several data types, including numbers and strings.
- ☐ All data values stored in `localStorage` are stored as strings.
- ☐ `localStorage` is used to store limited data sizes. It is usually restricted to just a few megabytes.

Question - 13

JSON data format

JSON uses a few strict rules to encode its content. Which of the following are true about those rules? Choose one or more.

- ☐ JSON uses double quotation marks to delimit strings.
- ☐ JSON uses single quotation marks to delimit strings.
- ☐ Trailing commas are forbidden.
- ☐ Trailing commas are allowed and recommended because of cleaner diffs when working with version control systems.

Question - 14

JavaScript Basics

JavaScript is perhaps the most widely used programming language for Web client purposes. All modern Web browsers use it. Which of the following are true?

- ☐ JavaScript is a strongly typed language. This implies that adding a number to a string is not possible and will result in a runtime error.
- ☐ When JavaScript code is executed in a browser, it uses client computational resources rather than server resources.
- ☐ JavaScript does not support recursion.
- ☐ Angular, React, and Vue are popular JavaScript frameworks used in Web development.

Question - 15

NPM and Bower

Which one of the following statements is true?

- ☐ Bower is an NPM package and can be installed using NPM.
- ☐ NPM is a Bower package and can be installed using Bower.
- ☐ Both NPM and Bower can be installed using each other.
- ☐ Bower and NPM aren't related and cannot be installed using each other.

Question - 16

NPM

What does NPM stand for?

- ☐ Node Package Manager
- ☐ Node Packet Manager
- ☐ Node Package Master
- ☐ Node Packet Master

Question - 17

Node Package Installation

What is the command to install a node package globally for all the users in the system?

- ☐ npm install -g package
- ☐ npm install --glob package
- ☐ npm install package
- ☐ npm_global_install install package

Question - 18

Find the File

In general, where is the package.json file present in any node application or module?

- ☐ Inside the node_modules folder
- ☐ In the root directory of the application/module
- ☐ Inside the packages folder

- ☐ Package.json is present in the /tmp folder

Question - 19

Meta Data

Which file holds the metadata for Grunt plugins?

- ☐ Gruntfile.js
- ☐ Gruntfile.coffee
- ☐ Package.json
- ☐ None of these

Question - 20

Method Requirements

If one task depends on the successful completion of another task (or tasks), what method can be used to force Grunt to abort if the other task didn't run, or if the other task failed?

- ☐ this.prerequisite
- ☐ this.successList
- ☐ this.dependencies
- ☐ this.requires

Question - 21

Install Grunt

How can you install Grunt in an Ubuntu Linux server?

- ☐ Using npm
- ☐ Using bower
- ☐ Using apt-get
- ☐ Using dpkg

Question - 22

Task Existence

How do you check if a task exists in Grunt?

- ☐ grunt.task.exists(name)

- ☐ grunt.task.present(name)
- ☐ grunt.task(name)
- ☐ grunt.task.task_exists(name)

Question - 23

Grunt Output III

```
grunt.initConfig({
  log: {
    foo: [1, 2, 3],
    bar: 'hello world',
    baz: false
  }
});

grunt.registerMultiTask('log', 'Log stuff.', function() {
  grunt.log.writeln(this.target + ': ' + this.data);
});
```

What is the output of the following Grunt task, when it is run as `grunt log`?

- ☐ foo: 1,2,3
- ☐ baz: false
- ☐ bar: hello world
- ☐ foo: 1,2,3 bar: hello world baz: false

Question - 24

Grunt Output II

```
grunt.initConfig({
  log: {
    foo: [1, 2, 3],
    bar: 'hello world',
    baz: false
  }
});

grunt.registerMultiTask('log', 'Log stuff.', function() {
  grunt.log.writeln(this.target + ': ' + this.data);
});
```

What is the output of the following Grunt task, when it is run as `grunt log:bar`?

- ☐ foo: 1,2,3
- ☐ bar: hello world
- ☐ foo: 1,2,3 bar: hello world baz: false
- ☐ baz: false

Question - 25

Grunt Output I

```
grunt.initConfig({
  log: {
    foo: [1, 2, 3],
    bar: 'hello world',
    baz: false
  }
});

grunt.registerMultiTask('log', 'Log stuff.', function() {
  grunt.log.writeln(this.target + ': ' + this.data);
});
```

What is the output of the following Grunt task, when it is run as `grunt log:foo`?

- ☐ foo: 1,2,3 bar: hello world baz: false
- ☐ foo: 1,2,3
- ☐ bar: hello world
- ☐ baz: false

Question - 26

Test Specification

In Grunt, if you don't specify a test to run, what is the name of the task that gets run by default (assuming the task exists)?

- ☐ Main
- ☐ Default
- ☐ Start
- ☐ Install

Question - 27

Negative Comparator

How do you use a negative comparator in Jasmine?

- ☐ .negative
- ☐ .not
- ☐ .negation
- ☐ .nor

Question - 28

Declared Variables

Variables declared in a describe are available to any *it* block inside the suite. True or False.

- ☐ True
- ☐ False

Question - 29

Pass the Test Case

```
// your applications custom code
function getValue( a, b ) {
    //YOUR CODE COMES HERE
};

// the Jasmine test code
describe("getValue(a, b) function", function() {
    it("should be equal", function(){
        expect( getValue(1, 2) ).toBe( 7 );
    });
    it("should be equal", function(){
        expect( getValue(7, 4) ).toBe( 29 );
    });
});
```

What should be the code at *//YOUR CODE COMES HERE*, so that the Jasmine test cases pass?

- ☐ return 2*a + 3*b;
- ☐ return 3*a + 2*b;
- ☐ return 3*a + 3*b;
- ☐ return 2*a + 2*b;

Question - 30

Execution Stop

In Jasmine, what is the configuration parameter to stop the execution of the script after the first failure of an expectation?

- ☐ stopOnFailure
- ☐ stopSpecOnExpectationFailure
- ☐ exitOnFailure
- ☐ exitSpecOnExpectationFailure

Question - 31

It Functions

How many `expect` can we have in an `it` function?

- ☐ 0
- ☐ 1
- ☐ 2
- ☐ An it function can have any number of expects.

Question - 32

Browser Display

Using JavaScript, check whether the browser is online. Display "*I am online*" if it is. Otherwise, display "*I am offline*".
If the browser is online, display the current *latitude* and *longitude* of your location.

```
<p id="status"></p>
<p id="location"></p>
```

The `<p>` with `id="status"` is used to display the status: "*I am offline*" or "*I am online*".

The `<p>` with `id="location"` is used to display the latitude and longitude in the format below:

Latitude: 12.9321589
Longitude: 77.61325529999999

- ☐

```
function checkStatus() {

    if(navigator.onLine){
        document.getElementById("status").innerHTML = "I am online";
        if (navigator.geolocation) {
            navigator.geolocation.getCurrentPosition(printLocation);
        }

    }
    else

        document.getElementById("status").innerHTML = "I am offline";
}
function printLocation(position) {
    document.getElementById("location").innerHTML = "Latitude: " + position.coords.latitude +
    "<br/>Longitude: " + position.coords.longitude;
}
```
- ☐

```
function checkStatus() {
```

```

    if(navigator.onLine){
        document.getElementById("status").innerHTML = "I am online";
        if (navigator.geolocation) {
            navigator.geolocation.getCurrentPosition(printLocation);
        }

    }
    else

        document.getElementById("status").innerHTML = "I am offline";
}

function printLocation(position) {
    document.getElementById("location").innerHTML = "Latitude: " + position.coords.latitude +
    "<br/>Longitude: " + position.coords.longitude;
}

☐ function checkStatus() {

    if(navigator.online){
        document.getElementById("status").innerHTML = "I am online";
        if (navigator.geolocation) {
            navigator.geolocation.getCurrentPosition(printLocation);
        }

    }
    else

        document.getElementById("status").innerHTML = "I am offline";
}
function printLocation(position) {
    document.getElementById("location").innerHTML = "Latitude: " + position.coords.latitude +
    "<br/>Longitude: " + position.coords.longitude;
}

☐ function checkStatus() {

    if(navigator.onLine){
        document.getElementById("status").innerHTML = "I am online";
        if (navigator.geolocation) {
            navigator.geolocation.getCurrentPosition(printLocation);
        }

    }
    else
        document.getElementById("status").innerHTML = "I am offline";
}
function printLocation(position) {
    document.getElementById("location").innerHTML = "Latitude: " + position.coords.latitude +
    "<br/>Longitude: " + position.coords.longitude;
}

```

Question - 33

JavaScript Visibility

A text taken from Wikipedia is given below. You have to toggle the visibility of paragraphs with `class="hide"` when the toggle button is clicked. Initially, all the paragraphs are displayed (see figure 1). When the toggle button is clicked, the visibility of elements with `<p class="hide">` is toggled.

jQuery is a cross-platform JavaScript library designed to simplify the client-side scripting of HTML.

jQuery is the most popular JavaScript library in use today, with installation on 65% of the top 10 million highest-trafficked sites on the Web. jQuery is free, open-source software licensed under the MIT License

jQuery's syntax is designed to make it easier to navigate a document, select DOM elements, create animations, handle events, and develop Ajax applications.

jQuery also provides capabilities for developers to create plug-ins on top of the JavaScript library. This enables developers to create abstractions for low-level interaction and animation, advanced effects and high-level, theme-able widgets.

Microsoft and Nokia bundle jQuery on their platforms.[7] Microsoft includes it with Visual Studio[8] for use within Microsoft's ASP.NET AJAX and ASP.NET MVC frameworks while Nokia has integrated it into the Web Run-Time widget development platform.[9] jQuery has also been used in MediaWiki since version 1.16.[10]

jQuery, at its core, is a DOM (Document Object Model) manipulation library. The DOM is a tree-structure representation of all the elements of a Web page and jQuery simplifies the syntax for finding, selecting, and manipulating these DOM elements. For example, jQuery can be used for finding an element in the document with a certain property (e.g. all elements with an h1 tag), changing one or more of its attributes (e.g. color, visibility), or making it respond to an event (e.g. a mouse click).

Toggle

Figure 1

jQuery's syntax is designed to make it easier to navigate a document, select DOM elements, create animations, handle events, and develop Ajax applications.

jQuery also provides capabilities for developers to create plug-ins on top of the JavaScript library. This enables developers to create abstractions for low-level interaction and animation, advanced effects and high-level, theme-able widgets.

Microsoft and Nokia bundle jQuery on their platforms.[7] Microsoft includes it with Visual Studio[8] for use within Microsoft's ASP.NET AJAX and ASP.NET MVC frameworks while Nokia has integrated it into the Web Run-Time widget development platform.[9] jQuery has also been used in MediaWiki since version 1.16.[10]

Toggle

Figure 2

The partial code is given below:

```
<script>
<!--Fill in the code here-->
</script>
<p class="hide">jQuery is a cross-platform JavaScript library designed to simplify the client-side scripting of HTML.</h2>

<p class="hide">jQuery is the most popular JavaScript library in use today, with installation on 65% of the top 10 million highest-trafficked sites on the
Web. jQuery is free, open-source software licensed under the MIT License</p>

<p>jQuery's syntax is designed to make it easier to navigate a document, select DOM elements, create animations, handle events, and develop Ajax
applications. </p>

<p class="visible">jQuery also provides capabilities for developers to create plug-ins on top of the JavaScript library. This enables developers to create
abstractions for low-level interaction and animation, advanced effects, and high-level, theme-able widgets. </p>

<p class="visible">Microsoft and Nokia bundle jQuery on their platforms.[7] Microsoft includes it with Visual Studio[8] for use within Microsoft's ASP.NET AJAX
and ASP.NET MVC frameworks while Nokia has integrated it into the Web Run-Time widget development platform.[9] jQuery has also been used in MediaWiki
since version 1.16.[10]</p>

<p class="hide">jQuery, at its core, is a DOM (Document Object Model) manipulation library. The DOM is a tree-structure representation of all the elements of
a Web page and jQuery simplifies the syntax for finding, selecting, and manipulating these DOM elements. For example, jQuery can be used for finding an
element in the document with a certain property (e.g. all elements with an h1 tag), changing one or more of its attributes (e.g. color, visibility), or making it
respond to an event (e.g. a mouse click).

</p>

<button >Toggle</button>

<script>
    $(document).ready(function(){
        $("button").click(function() {
            $("p.hide").Toggle();

        });
    });
</script>
```

- ```
$(document).ready(function(){
 $("button").click(function() {
 $("p.hide").toggle();
 });
});
```
- ☐ \$(document).ready(function(){  
 \$("button").click(function() {  
 \$("p.hide").toggle();  
  
 });  
});
- ☐ \$(document).ready(function(){  
 \$("button").click(function() {  
 \$("p.hide").Toggle();  
  
 });  
});

Question - 34

Code Prediction

Predict the output of the following code:

```
function Education() {
 this.education = "B.Tech";
}

function main() {
 Person = new Education();
 Person.name = "Michael";
 Person.surname = "Hicks";
 var Person;
 Person.residence = "Bangalore";
 Person.age = 23;
 Object.freeze(Person);
 console.log(delete Person.name);

 for (const item in Person) {
 console.log(item);
 }
}
```

- ☐ False education name surname residence age
- ☐ false education name surname residence age
- ☐ True education surname residence age
- ☐ true education surname residence age



- ☐ false education surname residence age
- ☐ true education name surname residence age
- ☐ Compile time error

### Question - 35

#### Array Element Passing

Examine the code snippet below. You have to identify the correct snippet for the main and pass each element of the array *list*=[0,4,1,3] such that for every number less than 2, the error message, "*num should be greater than 2*", is displayed.

Otherwise, the value returned from the function *twice* is displayed. What could be the possible solution(s)?

```
function Myfunc() {
 this.twice = function(num) {
 if (num < 2) {
 throw "num should be greater than 2"
 }
 return num * 2;
 }
}
```

☐

```
const myfunc = new Myfunc();
const list = new Array(0, 4, 1, 3);
for (let i = 0; i < 4; i++) {
 try {
 const res = myfunc.twice(list[i]);
 console.log(res);
 } catch (e) {
 console.log(e);
 }
}
```

☐

```
const myfunc = new Myfunc();
const list = {0, 4, 1, 3};
for (let i = 0; i < 4; i++) {
 try {
 const res = myfunc.twice(list[i]);
 console.log(res);
 } catch (e) {
 console.log(e);
 }
}
```

☐

```
const myfunc = new Myfunc();
const list = [0, 4, 1, 3];
for (let i = 0; i < 4; i++) {
 try {
 const res = myfunc.twice(list[i]);
 console.log(res);
 } catch (e) {
 console.log(e);
 }
}
```

☐

```
const myfunc = new Myfunc();
const list = [0, 4, 1, 3];
try {
 for (let i = 0; i < 4; i++) {
 const res = myfunc.twice(list[i]);
 console.log(res);
 }
} catch (e) {
 console.log(e);
}
```

### Question - 36

#### First Five Alerts

Select the option that has the correct sequence of first five alert messages, when the following javascript code snippet is run:

```
function func(x) {
 var z = 8;

 return function(y) {
 alert(x + y + z);
 }
}

var n1 = new Number(32);
var a1 = func(n1);
var n2 = new Number(16);
var a2 = func(n2);
var n3 = new Number(8);
var a3 = func(n3);
var n4 = new Number(4);
var a4 = func(n4);
var n5 = new Number(2);
var a5 = func(n5);

a1(2);
a2(4);
a3(8);
a4(16);
a5(32);
```

- ☐ 34, 36, 40, 48, 64
- ☐ 2, 4, 8, 16, 32
- ☐ 24, 24, 28, 28, 42
- ☐ 42, 28, 24, 28, 42

### Question - 37

#### Which of the following is not true of Javascript?

Which of the following is not true of Javascript?

- ☐ Javascript is case sensitive

- ☐ Semicolons at the end of Javascript statements are required
- ☐ Javascript statements can be grouped together in blocks
- ☐ Javascript is loosely typed

### Question - 38

What is the Purpose of the 'noscript' tag?

What is the purpose of the 'noscript' tag in javascript? Select the most accurate response

- ☐ Suppresses any script results that would be displayed on the web page
- ☐ Prevents scripts on the page from executing.
- ☐ Enclose text to be displayed by non-JavaScript browsers
- ☐ All of the above

### Question - 39

How to Capture Click Events in a Window

Which of the following javascript commands would capture all click events in a web page window?

- ☐ window.routeEvents(Event.CLICK);
- ☐ window.raiseEvents(Event.CLICK);
- ☐ window.handleEvents(Event.CLICK);
- ☐ window.captureEvents(Event.CLICK);

### Question - 40

querySelector

In Javascript, consider the following selector:

```
document.querySelector("p.five");
```

Which html elements would this query select in the document? Select the most accurate statement.

- ☐ The first five <p> elements
- ☐ The first <p> element with class='five'
- ☐ All <p> elements with class='five'
- ☐ None - this is not a valid selector query

### Question - 41

Which function would you use to find html elements using tag name?

Which function would you use to find html elements using tag name?

- ☐ document.getElementsByTagName()
- ☐ document.getElementsByTagName()
- ☐ document.getElementsUsingTagName()
- ☐ document.getElementUsingTagName()

#### Question - 42

What is the correct way to create a javascript array?

What is the correct way to create a javascript array?

- ☐ var items = ["Orange","Apple"];
- ☐ var items = {"Orange","Apple"};
- ☐ var items = new array("Orange","Apple");
- ☐ var items[] = {"Orange","Apple"};

#### Question - 43

What is the data type of null in Javascript?

What is the data type of *null* in Javascript?

- ☐ undefined
- ☐ integer
- ☐ object
- ☐ none

#### Question - 44

What is correct way to run a Javascript in "strict mode"?

What is correct way to run a Javascript in "strict mode"?

- ☐ <script mode=strict>  
</script>
- ☐ <script>  
"use strict"  
</script>

☐ `<script strictMode=True>`  
`</script>`

☐ `<script strict>`  
`</script>`

### Question - 45

Which of the following javascript functions perform the given tasks correctly?

Which of the following javascript functions perform the given tasks correctly?

- The function takes one numeric variable as parameter.
- If the variable is 0 then it creates a dialogue box which says "zero".
- If the variable is 1 then it creates a dialogue box which says "one".
- Otherwise it creates a dialogue box which says "Invalid input".

☐

```
const myFunction = value => {
 switch (value) {
 case 0:
 str = "0";
 break;
 case 1:
 str = "1";
 break;
 default:
 str = "Invalid input";
 break;
 }
 return alert(str);
}
```

☐

```
const myFunction = value => {
 switch (value) {
 case 0:
 str = "zero";
 case 1:
 str = "one";
 default:
 str = "Invalid input";
 }
 alert(str);
}
```

☐

```
const myFunction = value => {
 switch (value) {
 case "0":
 str = "zero";
 break;
 case "1":
 str = "one";
 break;
 default:
 str = "Invalid input";
 break;
 }
}
```

```
}
alert(str);
}

☐ const myFunction = value => {
 switch (value) {
 case 0:
 str = "zero";
 break;
 case 1:
 str = "one";
 break;
 default:
 str = "Invalid input";
 break;
 }
 alert(str);
}
```

### Question - 46

#### JavaScript Reserved Words

Which of the following are NOT JavaScript reserved words?

- ☐ throws
- ☐ auto
- ☐ throw
- ☐ struct

### Question - 47

#### What is 'this'?

In the global execution context (outside of any function), what does the keyword 'this' in javascript refer to? Select the most accurate answer.

- ☐ null
- ☐ the global object
- ☐ the web page
- ☐ the web browser

### Question - 48

#### JS Data Types

Which of the following is not a primitive Datatype in Javascript?

- ☐ Array

- ☐ Number
- ☐ String
- ☐ Boolean

### Question - 49

Value of x

Consider the following code. What would the line `console.log(x)` display below?

```
var x = 100;

function test() {
 if(false) {
 var x = 199;
 }
 console.log(x); //what does this log?
}

test();
```

- ☐ 100
- ☐ 199
- ☐ undefined
- ☐ null

### Question - 50

Equivalence Testing in JS

Which of the following is a difference between `==` and `===` in Javascript? Select the most accurate answer.

- ☐ `==` compares type and value. `===` compares type.
- ☐ `==` compares value. `===` compares type
- ☐ `==` compares value. `===` compares value and type
- ☐ `==` and `===` are equivalent

### Question - 51

Logging to the console

In what order will the numbers 1-4 be logged to the console when the code below is executed?

```
(function() {
 console.log(1);
 setTimeout(function(){console.log(2)}, 1000);
 setTimeout(function(){console.log(3)}, 0);
 console.log(4);
})();
```

- ☐ 1 2 3 4
- ☐ 4 3 2 1
- ☐ 1 4 3 2
- ☐ 1 3 4 2

### Question - 52

#### Screen Properties

Which of the following is not a screen property in Javascript?

- ☐ Height
- ☐ Width
- ☐ Selected
- ☐ ColorDepth

### Question - 53

#### Loops in JavaScript

Which of the following is not a looping structure in JavaScript?

- ☐ for
- ☐ while
- ☐ do - while
- ☐ do - if
- ☐ for each

### Question - 54

#### Which of the following is not a core datatype?

Which of the following is not a core datatype?

- ☐ Lists
- ☐ Dictionary
- ☐ Tuples



### Question - 55

The XML service(s) enable networking with javascript HTTP requests is:

The XML service(s) enable networking with javascript HTTP requests is:

- ☐ XMLHttpRequest
- ☐ XMLHttpRequest
- ☐ XMLHttpRequest
- ☐ All of the above

### Question - 56

What is the output?

What is the value of y after the following code?

```
let x;
let y = x === null;
```

- ☐ false
- ☐ true
- ☐ undefined
- ☐ null

### Question - 57

JavaScript: Appending Element to the Array

```
let arr = [1, 2, 3];
// complete the missing Javascript code on this line
console.log(arr); // outputs [1, 2, 3, 4]
```

- ☐ arr[arr.length + 1] = value
- ☐ arr[arr.length] = value
- ☐ arr[arr.length - 1] = value
- ☐ arr = arr + value

### Question - 58

What is the correct way to write a JavaScript array?

What is the correct way to write a JavaScript array?

- ☐ var txt = new Array("tim","kim","jim")
- ☐ var txt = new Array="tim","kim","jim"
- ☐ var txt = new Array:1=("tim")2=("kim")3=("jim")
- ☐ var txt = new Array(1:"tim",2:"kim",3:"jim")

### Question - 59

How can you add a comment in a JavaScript?

How can you add a comment in a JavaScript?

- ☐ //This is a comment
- ☐ REM This is a comment-->
- ☐ 'This is a comment
- ☐ #This is a comment

### Question - 60

Which of the following HTTP status codes denotes a server error?

Which of the following HTTP status codes denotes a server error?

- ☐ 100
- ☐ 200
- ☐ 400
- ☐ 500

### Question - 61

JavaScript Assignment

Consider the following JavaScript code snippet:

```
function foo () {
 return 5
}
```

What will the code `let myVar = foo; do?`

- ☐ Assign the integer value 5 to the variable *myVar*
- ☐ Assign a reference to the *foo* function to the variable *myVar*
- ☐ Throw an exception
- ☐ Nothing

### Question - 62

What is the correct way to have the function `checkState` called after 10 seconds?

What is the correct way to have the function `checkState` called after 10 seconds?

- ☐ `window.setTimeout(checkState, 10);`
- ☐ `window.setTimeout(checkState, 10000);`
- ☐ `window.setTimeout(checkState(), 10000);`
- ☐ `window.setTimeout(checkState(), 10);`

### Question - 63

What is the correct syntax to include a script named `myScript.js` into a page?

What is the correct syntax to include a script named `myScript.js` into a page?

- ☐ `<script href="myScript.js">`
- ☐ `<script name="myScript.js">`
- ☐ `<script src="myScript.js">`
- ☐ `<script root="myScript.js">`

### Question - 64

Which HTTP server does AJAX require?

Which HTTP server does AJAX require?

- ☐ Apache
- ☐ Microsoft web server
- ☐ Any HTTP server will work
- ☐ Sun application server

### Question - 65

Which of the following is the right option?

You want to update the following element with the XMLHttpRequest status. Which of the following approaches is correct for the purpose?

`<div id="statusCode"></div>`

- ☐ `var myDiv = document.getElementById ("statusCode"); myDiv.innerHTML = req.statusCode;`
- ☐ `var myDiv = document.getElementById ("statusCode"); myDiv.innerHTML = req.status;`
- ☐ `var myDiv = document.getElementById ("statusCode"); myDiv.setStatus (req.statusCode);`
- ☐ `var myDiv = document.getElementById ("statusCode"); myDiv.status = req.status;`

### Question - 66

What is the purpose of the NOSCRIPT tag?

What is the purpose of the NOSCRIPT tag?

- ☐ To prevent the page scripts from executing
- ☐ To shield a part of a page from being modified by JS, so that a statement like `aDiv.innerHTML = 'something'` has no effect.
- ☐ To enclose text to be displayed if the browser doesn't support JS
- ☐ There is no NOSCRIPT tag

### Question - 67

What should be called before 'send ()' to prepare an XMLHttpRequest object?

What should be called before 'send ()' to prepare an XMLHttpRequest object?

- ☐ `prepare ()`
- ☐ `open ()`
- ☐ `init ()`
- ☐ `build ()`